SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

This section "KitaVisa" introduces the strategic solution of the identified problem statement highlighted in the Product Requirement Documentation. The system aims to automate and intelligently orchestrate visa application workflows, addressing the fragmented and manual processes faced by international students, foreign workers, and first-time applicants.

1.1. Purpose:

The system analysis document highlights the technical scope and design-related decisions behind the KitaVisa development. The purpose is to transform unstructured visa-related user inputs into structured, actionable workflows, guided by AI-powered orchestration.

So, the key element of system that has been covered are as followed:

1. **Architecture**:
   The system operates as a web-based AI-assisted platform built using a client-server architecture.

   It is split into:
   a. Frontend (React dashboard + chat interface)
   b. Backend (Node.js API layer)
   c. AI Layer (Z.AI GLM reasoning engine)

   The GLM acts as the central intelligence layer that:
   a. interprets user input
   b. classifies visa intent
   c. supports confirmation-based decision flow
   d. generates requirement gaps after confirmation

   The backend acts as an orchestration layer that ensures:
   a. session tracking per user
   b. consistent API responses
   c. controlled flow between intent, confirmation, and analysis

2. **Data Flows:**
   a. Data flows from user inputs, such as free-text queries or document uploads, through the system where it is parsed, classified, and processed by the GLM.
   This input is sent to the backend /api/query endpoint, where:

   - The system stores session history
   - GLM is called to interpret intent

© UMHackathon 2026

- A single visa type is returned
- The system requests user confirmation

Once confirmed, the GLM analyzes missing documents for that specific visa type and structured requirements are returned to the frontend dashboard. The final output is a checklist-style workflow that the user follows step-by-step.

3. **Model Process:**

   The model represents an end-to-end conversational workflow and ensures the AI never directly finalizes actions without user confirmation.

   a. Unstructured input is received from user
   b. GLM extracts intent (single visa type only)
   c. System confirms with user before proceeding
   d. Upon confirmation, GLM performs gap analysis
   e. System generates structured visa checklist
   f. Frontend displays progress tracking dashboard

4. **Role of Reference:**

   This document serves as a reference for developers and testers to understand the full system flow, including:

   a. API contracts
   b. AI prompt behavior
   c. session-based workflow logic
   d. frontend-backend interaction rules

   It ensures consistency across implementation and testing stages.

1.2. **Background**:

The problem addressed is the fragmented and manual nature of the visa application process in Malaysia, where applicants have to navigate various government portals and informal sources of information. The solution aims to automate this process using AI, improving efficiency and accuracy for applicants and administrative staff alike.

Right now, visa applications in Malaysia are:

- scattered across multiple websites
- full of unclear requirements
- heavily dependent on manual interpretation

Most users don't actually know which visa category they fall under, what documents are required and whether they are even eligible yet. KitaVisa replaces that guessing process with a guided AI conversation.

© UMHackathon 2026

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
Visa Applicants	International students, foreign workers, first-time visa applicants, and dependent pass applicants who need assistance with their visa application process.	A simplified, personalized, and step-by-step workflow for each visa type (Social Visit, Student Pass, EP, TEP, Dependent Pass), with clear guidance on required documents and next steps.
Immigration staff	University administrative staff, HR departments, or other administrative personnel assisting with the visa application process for applicants.	A system to efficiently manage, track, and follow up on multiple visa applications for various applicants, ensuring deadlines are met and compliance is achieved.
Development Team	Building, deploying, and maintaining the platform.	A robust, scalable system architecture with well-documented code and clear API usage contracts.
QA Team	Validating the system behavior for all the user roles	Defined scope, the request and response formats, and coverage of edge cases

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	Web-based conversational visa assistant
Architecture	Client-server with AI orchestration layer

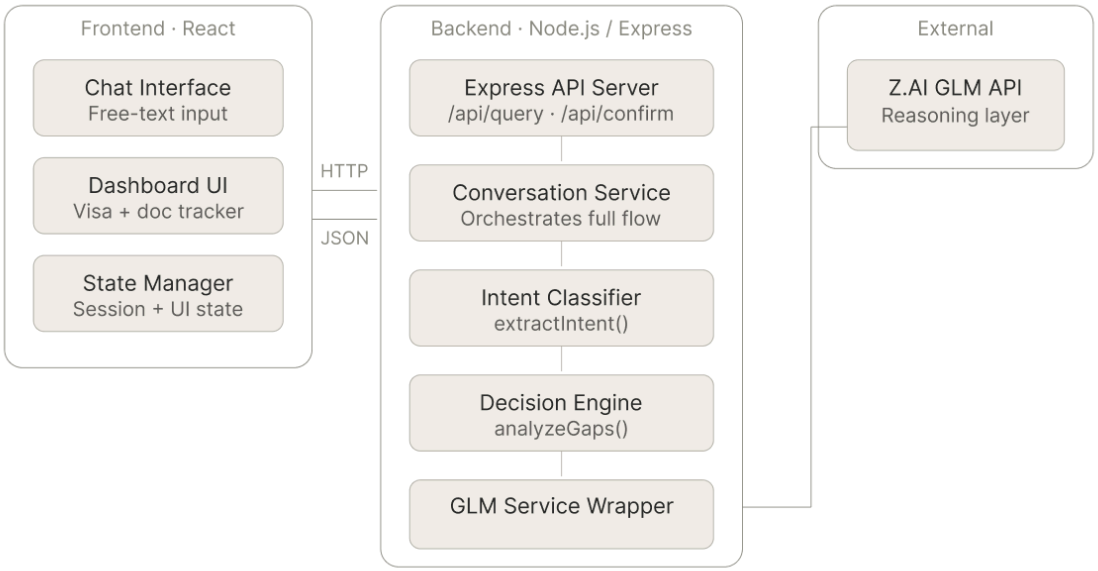
KitaVisa is structured as a client-server system where the frontend acts as the user interaction layer and communicates with the backend through defined API endpoints. The backend serves as the central orchestration layer, managing user sessions, controlling application flow, and coordinating all interactions with the GLM service. The AI layer is integrated as an external service, responsible for interpreting user input and generating structured outputs, while the backend ensures these outputs are validated and usable by the frontend. This separation allows the system to maintain consistent behavior while leveraging AI for reasoning tasks.

2.1.2 LLM as Service Layer

KitaVisa system integrates Z.AI's General Language Model (GLM) as a core service layer to handle the AI-driven decision-making and workflow orchestration. The GLM interprets unstructured data such as free-text queries and uploaded documents, categorizes visa types, identifies missing information, and dynamically orchestrates the application workflow.

2.1.3 Dependency Diagram

© UMHackathon 2026



The KitaVisa system is structured around a lightweight backend orchestration layer that coordinates user interactions, AI reasoning via GLM, and workflow decision-making. Instead of relying on a traditional monolithic flow, the system separates responsibilities into logical services that interact through well-defined API calls.

At the core of the system is the GLM, which acts as the reasoning engine. However, the GLM is strictly controlled through prompt engineering and schema-constrained outputs. Every request sent to GLM is constructed by the backend using predefined system instructions that define the expected visa categories, decision rules, and strict JSON output format. The model is only responsible for interpretation and structured reasoning.

The backend (Node.js Express server) acts as the orchestration layer between the frontend and GLM. It manages session handling, routes requests, applies intent classification logic, and decides which GLM function to call. The system currently uses three main GLM-driven services: intent extraction, visa classification, and gap analysis. These services are isolated functions that ensure each AI call has a single responsibility and predictable output format.

The frontend (React + Vite) interacts with the backend through two primary API endpoints:

- /api/query for initial user input and visa identification
- /api/analyze-gaps for generating missing document requirements after confirmation

© UMHackathon 2026

The frontend does not directly communicate with GLM. Instead, it only consumes structured responses from the backend, such as visa type recommendations, confirmation prompts, and gap lists.

When a user submits a message, the system processes it in the following sequence:

a. The frontend sends the raw user input and sessionId to the backend (/api/query).
b. The backend passes the input to the Intent Classifier (GLM-powered).
c. The classifier extracts a single visa intent (Student, Work, Dependent, Social Visit, or Other).
d. The result is passed to the decision layer, which prepares a confirmation response.
e. If the user confirms, the backend triggers analyze gap.
f. The GLM receives a structured prompt containing:
    - selected visa type
    - predefined document requirements
    - user input context
g. GLM returns a structured JSON response containing missing requirements (gaps).
h. The backend validates and forwards the structured output to the frontend dashboard.

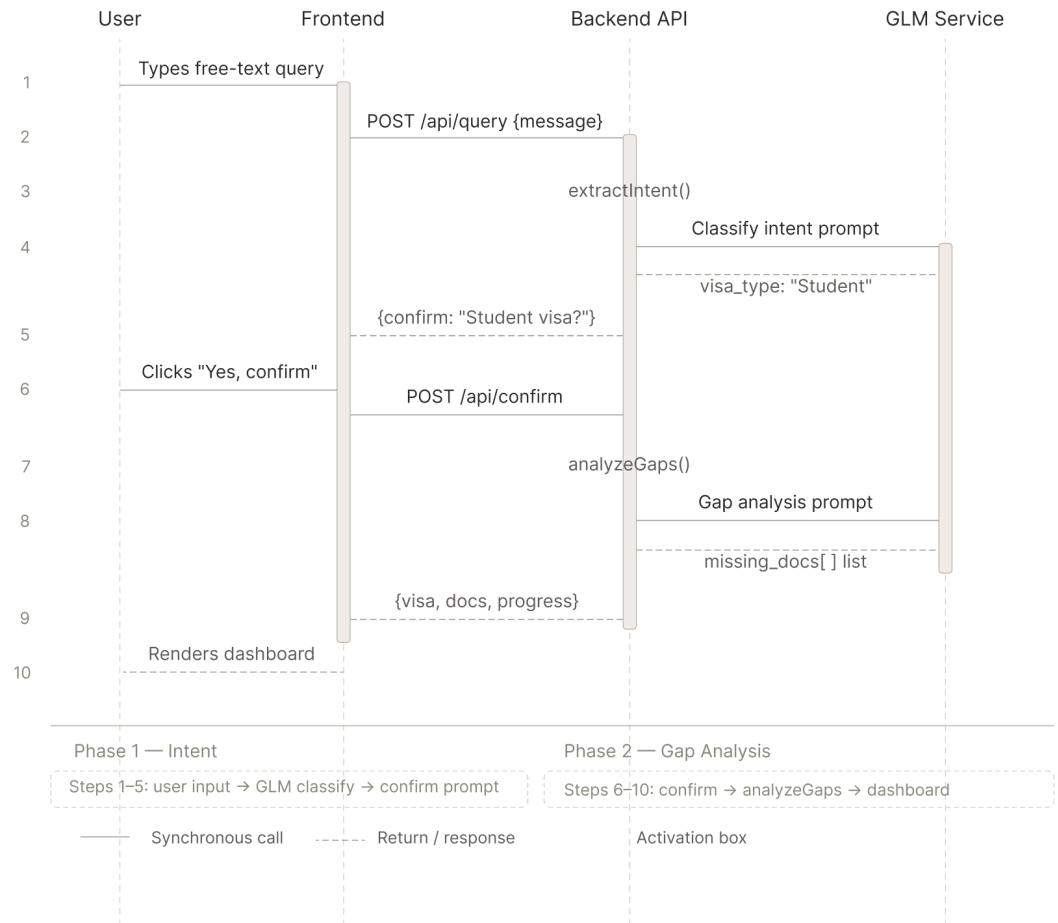
The GLM context window is intentionally minimized to improve reliability and reduce token cost. Only the following data is included in each request:

- Latest user input (or confirmed input)
- System prompt defining visa rules and output format
- Selected visa type (for gap analysis only)

The dependency structure is intentionally designed with clear separation of responsibilities across the system. The frontend is purely presentational, meaning it only handles user interaction and UI rendering without containing any business logic or decision-making capability. All processing logic is handled by the backend, which acts as the single source of orchestration for the entire system. It manages user sessions, routes requests, coordinates service calls, and determines when and how the GLM should be invoked. The GLM itself functions as a stateless reasoning engine, used only for structured inference tasks such as intent classification and gap analysis, based on carefully engineered prompts and constrained JSON outputs. It does not maintain memory or conversation context, ensuring consistent and predictable behavior across requests. As a result, the overall system logic remains deterministic and is not dependent on free-form conversational behavior. This strict separation of concerns ensures that the system is scalable, easier to debug, and remains stable even when handling complex visa classification and multi-step workflow scenarios.

© UMHackathon 2026

2.2.2. Sequence Diagram



User Interaction Flow

The process begins when a user submits a natural language request such as "I got accepted into Universiti Malaya and need help with my visa". This triggers the system's core pipeline.

a. User ⟶ Frontend (React UI)
   The user enters a message into the chat interface. The frontend captures this input along with a session ID and sends it to the backend via the /api/query endpoint.
b. Frontend ⟶ Backend (/api/query)
   The backend receives the request and immediately forwards the input to the Intent Classification Service. At this stage, no AI response is returned to the frontend yet — only processing begins.
c. Backend ⟶ GLM (Intent Extraction)
   The backend constructs a structured prompt and sends it to the GLM API. The

© UMHackathon 2026

GLM analyzes the user's input and returns a single visa intent (e.g., Student Pass).

d. Backend → Decision Engine
The extracted intent is passed into the decision layer, which maps it to a valid visa category and prepares a structured response. This step ensures the system remains rule-based and consistent even if user input is ambiguous.

e. Backend → Frontend (Confirmation Response)
The backend responds to the frontend with:

- detected visa type
- confirmation message
- recommendation array

f. At this stage, the system pauses and waits for user confirmation before proceeding further.

Confirmation Step

a. User → Frontend (Confirmation Action)
The user either confirms the recommendation or requests clarification. This ensures that the system does not prematurely proceed with incorrect assumptions.

b. Frontend → Backend (/api/analyze-gaps)
Once confirmed, the frontend sends the selected visa type back to the backend to trigger the next stage of processing.
    - Gap Analysis & Workflow Generation

c. Backend → GLM (Gap Analysis Request)
The backend sends a structured prompt to GLM containing:
    - confirmed visa type
    - predefined document requirements
    - user input context

d. GLM → Backend (Structured Gap Response)
The GLM returns a JSON response listing missing documents or requirements (gaps), based on the selected visa category.

e. Backend → Frontend (Workflow Output)
The backend formats and sends the final structured response to the frontend. This includes:

f. missing documents

g. severity levels

h. guidance messages

© UMHackathon 2026

Email: umhackathon@um.edu.my

i. Frontend → User (Dashboard Display)
The frontend renders the visa workflow dashboard, showing step-by-step requirements and progress tracking for the user.

Key System Features Demonstrated

a. Intent Classification: Converts unstructured text into a structured visa category
b. Confirmation-Based Control Flow: Prevents premature workflow execution
c. AI-Powered Gap Analysis: Identifies missing documents dynamically using GLM
d. Stateful Session Handling: Maintains user progress across steps
e. Structured Output Enforcement: Ensures all AI responses are valid JSON for system reliability

2.2. Technological Stack

2.2.1. Frontend: React + Vite
2.2.2. Backend: Node.js + Express
2.2.3. AI Engine: Z.AI GLM
2.2.4. Deployment: Local / cloud-ready architecture

2.3. Key Data Flows

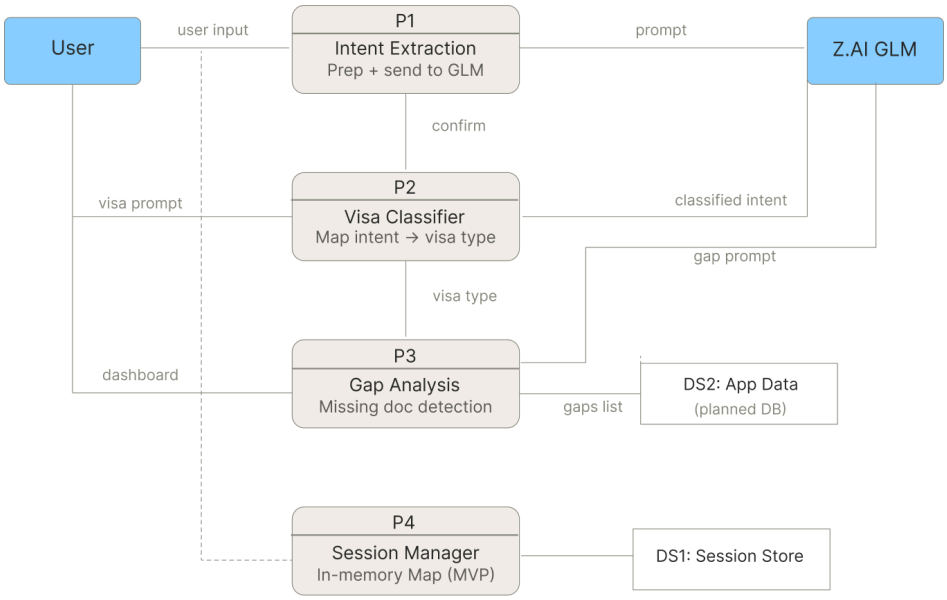
2.3.1. Here, do show the how data are moving through the system and how its structure and stored

2.3.1.1. Data Flow Diagram (DFD)

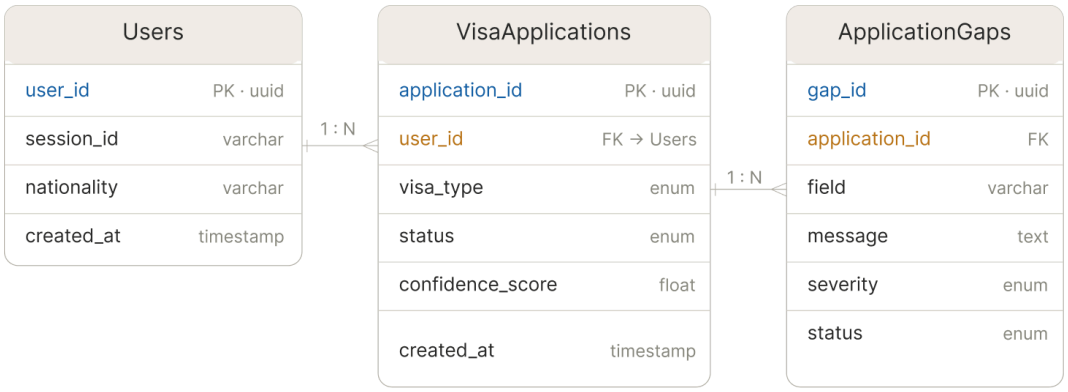
Level 0 DFD

© UMHackathon 2026

Level 1 DFD



2.3.2. Normalized Database Schema



- Users 1:N VisaApplications: One user may have multiple visa attempts
- VisaApplications 1:N ApplicationGaps: Each application surfaces multiple missing documents
- visa_type enum: Student | Work | Dependent | Social Visit

Current MVP uses in-memory Map keyed by session_id. This schema is future-ready for PostgreSQL with no structural API changes needed.

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

3.1.1. Do list down 3-5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026. **[e.g. in Table]**

#	Features	Description
1	Visa Workflow Generation	The system generates a personalized, multi-step workflow for each user based on their visa type and specific needs.
2	Document Parsing & Extraction	The system automatically parses uploaded documents (e.g., offer letters) to extract key details.
3	Progress Tracking	The system tracks the user's progress through various stages of the visa application process.

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality	Requirements	Implementation
Scalability	The system must handle a high volume of visa application workflows, especially during peak usage times.	[e.g. Use AWS ASG for scaling Servers and ELB for distribution of traffic.]
Reliability	The system must ensure that visa application workflows are reliable, with no data loss during interruptions, and user data is protected.	[e.g., payment confirmation triggers database INSERT SQL query for orders.]
Maintainability	Each service like (Visa Workflow Service, Document Parsing, etc) is separated and deployed individually.	[e.g. Independent Pipeline for each service during CI/CD stage with its own configurations.]
Token Latency	The API calls to GLM must return a response within 3s under normal load but at p95 at peak time.	[e.g. Timeout of 5s for Async API calls. Displaying user-facing loading state while awaiting for response]

© UMHackathon 2026

Cost Efficiency	The system's token usage for GLM API calls must not exceed a defined budget threshold per user session.	[e.g. The length of the prompts should be fixed length. The inputs from the user are truncated or chunked before injecting and the repeating queries are cached for 24 hours.]

3.3. Out of Scope / Future Enhancements

a. Integration with Government and University Portals
The system does not connect to official immigration systems or university databases. It does not submit applications, verify documents, or retrieve official data.

b. Mobile Native Application
The system is web-based only and does not include native iOS or Android applications.

c. Real-Time Visa Status Tracking
The system does not track or display live application status from immigration authorities. It only provides guidance before submission.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. **Grayscale Rollout & A/B Testing**: Do highlight team's strategy of deployment [e.g. new feature of user-preferred food menu are released to 5% of real users initially, monitoring and then gradually increased until 100% if no issues are detected. After it passes, A/B testing is conducted to determine which variants of two strategies perform better.]

4.1.2. **Emergency Rollback (ER) & Golden Release**: Do mentioned the strategies for Emergencies. [e.g. In terms of recovery & traffic management for YFood system, if there are anomalies, ER is triggered. The automated pipeline does deploy the Golden Version of the application.]

© UMHackathon 2026

4.1.3. Priority Matrix:

Priority	Condition (Trigger)	Action
P1 – Critical	Backend crashes, API endpoints not responding, or GLM API fails consistently	Immediate rollback or restart of backend service. Disable affected endpoint if needed and fall back to safe default response (e.g. “Please try again”).
P2 – High	Incorrect visa classification or /api/analyze-gaps returns empty/invalid JSON repeatedly	Log error, fix parsing or prompt issue, and redeploy. Ensure consistent JSON output before continuing usage.
P3 – Medium	Slow response time (>20s seconds from GLM), occasional timeout	Retry request automatically, show loading state on frontend, and monitor frequency of delays.
P4 – Low	Minor UI inconsistencies, unclear chatbot responses, or non-blocking UX issues	Log for improvement, fix after core functionality is stable. Does not block system usage.

4.2. **Monitoring**:

The monitoring plan for KitaVisa focuses on maintaining system stability by detecting failures in API availability, AI responses, and data consistency. The approach is lightweight and primarily relies on backend validation and logging.

At the API level, the system monitors the availability and correctness of the two main endpoints: `/api/query` and `/api/analyze-gaps`. If an endpoint fails to respond, returns a server error, or produces unexpected output, it is treated as a critical issue. In such cases, the backend service is restarted or reverted to a previously stable state to restore functionality.

For AI interactions, all responses from the GLM are validated before being returned to the frontend. The system checks for empty outputs, malformed JSON, and missing required fields. If any of these conditions occur, the response is replaced with a safe fallback (e.g. `"visaType": "Unknown"` or an empty gaps array), and the error is logged. Repeated occurrences indicate an issue with prompt design or response handling and require further debugging.

© UMHackathon 2026

Response time is also monitored to ensure acceptable system performance. If AI responses exceed a defined threshold (e.g. 5–10 seconds), the system retries the request and maintains a loading state on the frontend. Persistent delays are recorded and treated as performance issues that may require adjustment of timeout settings or request handling.

Backend logging serves as the primary monitoring mechanism. It records API errors, AI failures, parsing issues, and timeout events. These logs are used to trace the source of failures and support debugging during development and testing.

In addition, the system implicitly monitors consistency between backend responses and frontend expectations. Missing or incorrectly structured fields indicate a breakdown in the API contract and trigger investigation.

Overall, the monitoring approach emphasizes early detection of failures, controlled fallback behavior, and consistent API responses rather than complex infrastructure-level monitoring.

4.3. Assumptions:

a. **Stable Internet Connectivity**
It is assumed that users have a reasonably stable internet connection when interacting with the system. Since the platform relies on real-time API calls (particularly to the GLM service), intermittent or slow connections may result in delayed responses or failed requests. The system does not currently include offline support or advanced retry mechanisms beyond basic request handling.

b. **Access to Web-Based Interface**
All users, including visa applicants, university staff, and administrative personnel, are assumed to have access to a web browser on a desktop or mobile device. The system is designed as a web-based application and does not provide native mobile app support.

c. **User-Provided Information is Accurate**
The system assumes that users provide truthful and reasonably accurate information when interacting with the chatbot. Since visa classification and gap analysis depend entirely on user input, incomplete or misleading information may lead to incorrect recommendations or missing requirements.

d. **GLM API Availability and Stability**
The system assumes that the external GLM API is available and responsive within acceptable latency limits. Core functionalities such as intent classification and gap analysis depend on this service. Any downtime or degradation in the API may affect system performance.

e. **Single User per Session Context**
Each session is assumed to represent a single user and a single visa application

© UMHackathon 2026

context. The system does not currently support multi-user collaboration within the same session or switching between multiple applicants.

4.4. External Dependencies:

Tools	Purpose	Risks
Z Ai GLM API	The KitaVisa system relies on Z.AI's GLM to classify visa types, detect missing information, and generate personalized workflows.	If there are issues with GLM's availability or performance, the visa workflow automation could be disrupted.

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline:

Task	Day 1	Day 2	Day 3	Day 4	Day 5	Day 6	Day 7	Day 8	Day 9	Day 10
Planning										
Research & Analysis										
Project development										
Testing & Debugging										
Final review & Submission										

5.2. Team Members Role:

Cita Wafa Atiah: Backend & AI Integration

Responsible for designing and implementing the backend system, including API endpoints (/api/query, /api/analyze-gaps) and overall request flow. Handles integration with the GLM API, including prompt design, response parsing, and error handling. Also manages session logic, intent classification, and consistent JSON responses for frontend consumption.

Sheeva Puti Utsula: Frontend & User Experience

Responsible for building the web interface using React, including the chatbot interaction flow, confirmation UI, and results display (identified gaps dashboard). Ensures that the user experience is intuitive, especially around unclear inputs, confirmation steps, and navigation. Work with backend integration to ensure smooth data flow and correct rendering of responses.

© UMHackathon 2026

5.3. Recommendations:

a. Introduce a lightweight caching layer (e.g., Redis) to store repeated AI responses or session states, reducing API calls and latency.
b. Add structured logging for backend requests and AI responses to make debugging easier during demos.
c. Implement basic validation and fallback handling for AI outputs to prevent UI breaks when responses are malformed.
d. If scaling beyond prototype, separate AI service logic into its own service layer to improve maintainability.

© UMHackathon 2026